

Instituto Politécnico Nacional

Centro de Investigación en Computación (CIC)

Departamento de diplomados y extensión profesional

Programación Python con Aplicaciones en el Ámbito Científico

Profesor Alan Badillo Salas

Tarea 2

Bibliotecas científicas de python

Semana 2. Viernes 11, septiembre 2026.

Introducción

En la segunda sesión del curso comenzamos a avanzar con el uso de bibliotecas científicas, esto nos permitirá empezar a aplicar la programación mediante análisis estadístico para poder sentar las bases de los siguientes temas en análisis de datos y visualización de datos.

Para instalar las librerías abre una terminal y escribe:

```
python -V

python -m pip install numpy
python -m pip install pandas
python -m pip install matplotlib
python -m pip install seaborn
```

En caso de que el primer comando de la versión falle, instala correctamente python desde <https://python.org>.

En esta **Tarea 2**, entenderemos la instalación de los paquetes científicos más importantes y el manejo de Arreglos de Numpy, Series de Pandas y las gráficas principales de Matplotlib y Seaborn.

Lee atentamente cada problema y resuélvelo. En un documento **PDF** con una portada breve la identidad del *Instituto Politécnico Nacional*, el *Centro de Investigación en Computación* y el *Departamento de diplomados y extensión profesional*, con tu nombre completo, correo, así como lugar y fecha, documenta los resultados de la tarea y envíalos por correo a **badillosalas@outlook.com** con el título **PyCien 2026 - Tarea 2**. No es necesario poner cuerpo en el correo o adjuntar más archivos, solo el documento PDF es suficiente. El documento PDF se debe llamar *pycien26.SIGLAS.pdf* con las SIGLAS formadas por tu nombre y apellidos.

1. Problema 1 - Arreglos de Numpy

El siguiente código muestra la construcción de un arreglo de edades de Numpy, este permite obtener los indicadores principales de los datos mediante algunas funciones incorporadas al arreglo, analiza el siguiente código y ejecútalo:

```
import numpy

edades = numpy.array([23, 25, 29, 28, 27, 34, 45, 19, 32, 22])

print(edades)
print(edades.size)
print(edades.sum())
print(edades.mean())
print(edades.var())
print(edades.std())
print(edades.min())
print(edades.max())
```

Ahora analiza el código cuando se tiene un formato construido por un texto f-string multilinea usando triple comilla doble:

```
import numpy

edades = numpy.array([23, 25, 29, 28, 27, 34, 45, 19, 32, 22])

print(f"""
EDADES:      {edades}
-----
TOTAL:       {edades.size:34d}
-----
SUMA:        {edades.sum():34.3f}
PROMEDIO:    {edades.mean():34.3f}
-----
VARIANZA:    {edades.var():34.3f}
DESV. EST.:  {edades.std():34.3f}
-----
MÍNIMO:      {edades.min():34d}
MÁXIMO:      {edades.max():34d}
""")
```

Contesta las siguientes preguntas:

1. ¿Cuál es la suma de las edades?
2. ¿Cuál es el promedio de las edades?
3. ¿Cuál es la varianza de las edades? ¿Qué significa?

4. ¿Cuál es la desviación estándar de las edades? ¿Qué significa?
5. ¿Cuál es la edad mínima?
6. ¿Cuál es la edad máxima?
7. ¿Cómo cambian los datos si se borra la edad de 45 del arreglo de las edades?

2. Problema 2 - Series de pandas

Ahora, en este segundo problema se construirá una serie de Pandas en lugar de un arreglo de Numpy para procesar mejor los datos y sacar más estadísticos fácilmente, analiza el siguiente código y ejecútalo:

```
import pandas

edades = pandas.Series([23, 25, 29, 28, 27, 34, 19, 32, 22])

print(f"""
EDADES:      {edades}
-----
TOTAL:       {edades.count():34d}
-----
SUMA:        {edades.sum():34.3f}
PROMEDIO:    {edades.mean():34.3f}
-----
VARIANZA:    {edades.var():34.3f}
DESV. EST.:  {edades.std():34.3f}
-----
MÍNIMO:      {edades.min():34d}
MÁXIMO:      {edades.max():34d}
-----
Q1 (25%):    {edades.quantile(0.25):34.3f}
Q2 (50%):    {edades.quantile(0.50):34.3f}
Q3 (75%):    {edades.quantile(0.75):34.3f}
-----
2.5%:        {edades.quantile(0.025):34.3f}
5%:          {edades.quantile(0.05):34.3f}
95%:         {edades.quantile(0.95):34.3f}
97.5%:       {edades.quantile(0.975):34.3f}
""")
```

Ahora, analiza el código que le pide a la serie de Pandas describir los estadísticos principales:

```
import pandas

edades = pandas.Series([23, 25, 29, 28, 27, 34, 19, 32, 22])

print(edades)
print("-" * 40)
print(edades.describe())
```

Contesta las siguientes preguntas:

1. ¿Cuál es la suma de las edades?
2. ¿Cuál es el promedio de las edades?
3. ¿Qué significa el cuartil 1 (Q_1 al 25%)?
4. ¿Qué significa el cuartil 2 (Q_2 al 50 % o mediana)?
5. ¿Qué significa el cuartil 3 (Q_3 al 75%)?
6. ¿Qué significa el percentil al 2,5 %?
7. ¿Qué significa el percentil al 5 %?
8. ¿Qué significa el percentil al 95 %?
9. ¿Qué significa el percentil al 97,5 %?
10. ¿Qué porcentaje de la población tiene más de 29 años?

3. Problema 3 - Graficar puntos X, Y con Matplotlib

El siguiente código muestra cómo graficar, guardar y mostrar la gráfica de puntos (x, y) definida:

```
import numpy
from matplotlib import pyplot

x = numpy.array([1, 3, 6, 7, 9])
y = numpy.array([4, 5, 20, 16, 11])

pyplot.plot(x, y)
pyplot.savefig("g1.png")
pyplot.show()
```

El siguiente código agrega estilos personalizados a la gráfica, como el tipo de marcador, el tipo de línea, el color principal y el color de los marcadores:

```
import numpy
from matplotlib import pyplot

x = numpy.array([1, 3, 6, 7, 9])
y = numpy.array([4, 5, 20, 16, 11])

pyplot.plot(x, y,
            marker="s", linestyle="--",
            color="red", markerfacecolor="blue")
pyplot.savefig("g2.png", dpi=300)
pyplot.show()
```

El siguiente código muestra cómo graficar, guardar y mostrar la gráfica del seno y del coseno de 20 puntos entre $-\pi$ y π usando matplotlib:

```

import numpy
from matplotlib import pyplot

x = numpy.linspace(-numpy.pi, numpy.pi, 20)

y1 = numpy.sin(x)
y2 = numpy.cos(x)

pyplot.plot(x, y1,
             marker=".", linestyle="--",
             color="pink", markerfacecolor="red")
pyplot.plot(x, y2,
             marker=".", linestyle="--",
             color="skyblue", markerfacecolor="blue")
pyplot.savefig("g3.png", dpi=300)
pyplot.show()

```

Contesta las siguientes preguntas:

1. ¿Cuál gráfica tiene mayor resolución *g1.png* o *g2.png*?
2. Investiga cómo colocarle un título y leyendas a los ejes en la gráfica y ajusta las gráficas para que sean más explicativas
3. Investiga 3 tipos de marcadores y 3 tipos de estilos de línea y genera las gráficas

4. Problema 4 - Histogramas con Seaborn

El histograma divide el eje de análisis en intervalos (*bins*) iguales y muestra la frecuencia que cae en cada intervalo. Analiza el siguiente código que muestra el histograma de la serie de precios:

```
import pandas
from matplotlib import pyplot
import seaborn

precios = pandas.Series([
    22.5, 22.9, 26.5, 21.45,
    21.5, 25.9, 27.5, 24.45,
    23.5, 26.9, 25.5, 26.45,
    24.5, 24.9, 24.5, 24.45,
    27.5, 27.9, 28.5, 21.45,
])

seaborn.histplot(x=precios)
pyplot.show()
```

Ahora analiza la variación en el número de bins a 10:

```
import pandas
from matplotlib import pyplot
import seaborn

precios = pandas.Series([
    22.5, 22.9, 26.5, 21.45,
    21.5, 25.9, 27.5, 24.45,
    23.5, 26.9, 25.5, 26.45,
    24.5, 24.9, 24.5, 24.45,
    27.5, 27.9, 28.5, 21.45,
])

seaborn.histplot(x=precios, bins=10)
pyplot.show()
```

La densidad estimada (*KDE - Kernel Density Estimation*) representa la masa continua ideal que debería distribuirse a lo largo del eje de análisis según los datos disponibles. Analiza la gráfica que estima la densidad de los precios:

```
import pandas
from matplotlib import pyplot
import seaborn

precios = pandas.Series([
    22.5, 22.9, 26.5, 21.45,
    21.5, 25.9, 27.5, 24.45,
    23.5, 26.9, 25.5, 26.45,
    24.5, 24.9, 24.5, 24.45,
    27.5, 27.9, 28.5, 21.45,
])

seaborn.kdeplot(x=precios, fill=True)
pyplot.show()
```

Contesta las siguientes preguntas:

1. ¿Cómo explicas lo que dice el histograma?
2. ¿Cómo cambia el histograma si aumentamos a 20 *bins*? ¿Es útil?
3. Con pocos datos, ¿Es más útil el KDE o el Histograma?
4. ¿Dónde se acumula más densidad de datos sobre el eje de los precios?

5. Problema 5 - Gráfica de caja con Seaborn

La gráfica de caja permite resumir la densidad de un eje de análisis de manera sencilla usando la información de los cuartiles Q_1 , Q_2 y Q_3 . Analiza el código que genera la gráfica de caja para los precios:

```
import pandas
from matplotlib import pyplot
import seaborn

precios = pandas.Series([
    22.5, 22.9, 26.5, 21.45,
    21.5, 25.9, 27.5, 24.45,
    23.5, 26.9, 25.5, 26.45,
    24.5, 24.9, 24.5, 24.45,
    27.5, 27.9, 28.5, 21.45,
])

seaborn.boxplot(x=precios, fill=False)
pyplot.show()
```

Contesta las siguientes preguntas:

1. Visualmente, ¿Cuál es el precio mínimo?
2. Visualmente, ¿Cuál es el precio máximo?
3. Visualmente, ¿Cuál es el precio mediano?
4. Visualmente, ¿Cuál es el precio al 25 % de los datos?
5. Visualmente, ¿Cuál es el precio al 75 % de los datos?
6. Visualmente, ¿Dónde se concentra el 50 % de los precios observados?